

Álgebra Lineal Computacional con Python

Fundamentos Matemáticos, Computación Numérica y Machine Learning

*Manual de referencia integral y avanzado sobre la formulación matemática y la implementación computacional con **NumPy** y **SciPy**: vectores, transformaciones lineales, sistemas de ecuaciones, teoría espectral, factorizaciones matriciales y aplicaciones directas a la ciencia de datos.*

Ecosistema Científico y Entorno Tecnológico

NumPy 1.2x/2.x · SciPy 1.1x · BLAS / LAPACK

Incluye sintaxis moderna con operador infijo @, np.einsum y factorizaciones avanzadas

Manual Técnico Avanzado · 17 de septiembre de 2026

Índice

1	Introducción y Arquitectura Numérica	2
1.1	Diferencias Estructurales: Listas Nativas vs. <code>ndarray</code>	2
1.2	Organización en Memoria: Disposición C vs. Fortran	2
1.3	Reglas de Difusión (<i>Broadcasting</i>)	2
2	Vectores y Espacios Vectoriales	3
2.1	Definición Matemática y Representación Computacional	3
2.2	Independencia Lineal y Combinación Lineal	3
2.3	Producto Escalar (Producto Punto)	4
2.4	Familias de Normas Vectoriales (L_1 , L_2 , L_∞)	4
2.5	Distancia Euclidiana y Similitud del Coseno	4
2.6	Proyección Ortogonal de un Vector	5
2.7	Producto Cruz en $\mathbb{R}^3$	5
3	Matrices y Álgebra Matricial	5
3.1	Tipología y Familias de Matrices	5
3.2	Multiplicación Matricial, Producto Exterior y de Kronecker	6
3.3	Traza y sus Propiedades Cíclicas	6
3.4	Notación de Suma de Einstein con <code>np.einsum</code>	6
4	Transformaciones Lineales 2D/3D	6
4.1	Definición de Operador Lineal	6
4.2	Matrices de Transformación Clásicas en 2D	7
5	Determinante, Rango y Subespacios	8
5.1	Determinante e Interpretación Geométrica	8
5.2	Los Cuatro Subespacios Fundamentales de Strang	8
5.3	Inversa Clásica y Pseudoinversa de Moore-Penrose	9
6	Sistemas Lineales y Análisis Numérico	9
6.1	Formulación Matricial y Teorema de Rouché-Frobenius	9
6.2	Número de Condición y Estabilidad Numérica	10
6.3	Sistemas Sobredeterminados y Mínimos Cuadrados (OLS)	10
7	Autovalores y Diagonalización	11
7.1	Definición y Polinomio Característico	11
7.2	Diagonalización de Matrices	11
7.3	El Teorema Espectral para Matrices Simétricas	11
7.4	Matrices Definidas Positivas	12
8	Descomposiciones Matriciales	12
8.1	Descomposición LU con Pivoteo Parcial ($PA = LU$)	12
8.2	Descomposición de Cholesky ($A = LL^T$)	13
8.3	Descomposición QR ($A = QR$)	13
8.4	Descomposición en Valores Singulares (SVD)	14
9	Aplicaciones en ML y Data Science	14
9.1	Análisis de Componentes Principales (PCA) desde Cero	14
9.2	Algoritmo PageRank: Autovectores en Grafos Web	15
10	Tabla Resumen y Complejidad	17
11	Buenas Prácticas y Conclusiones	18

1. Introducción y Arquitectura de la Computación Numérica

El álgebra lineal es el lenguaje fundamental sobre el que descansan la inteligencia artificial, el aprendizaje profundo (*Deep Learning*), el procesamiento digital de señales, los motores gráficos 3D, la visión computacional y la optimización numérica.

En Python, la biblioteca canónica para computación matricial es **NumPy** (*Numerical Python*), complementada por las rutinas especializadas de **SciPy** (`scipy.linalg`).

1.1. Diferencias Estructurales: Listas Nativas vs. `ndarray`

Las listas nativas de Python son colecciones dinámicas de punteros dispersos en memoria hacia objetos genéricos. Cada cálculo sobre una lista requiere comprobaciones de tipos en tiempo de ejecución e indirección de memoria (*boxing/unboxing*).

Por el contrario, el arreglo multidimensional `np.ndarray` asigna un **bloque contiguo de memoria** con un tipo de dato homogéneo (`float64`, `int32`, etc.). Esto posibilita:

- **Vectorización:** Ejecución de bucles a nivel de C/Fortran sin la sobrecarga del intérprete de Python.
- **Aceleración SIMD:** Aprovechamiento de instrucciones de hardware (*Single Instruction, Multiple Data*, como AVX-512).
- **Enlace con BLAS y LAPACK:** Delegación directa de operaciones a bibliotecas hiper-optimizadas (OpenBLAS, Intel MKL).

```
1 import numpy as np
2 import scipy.linalg as la
3
4 # Verificación del backend BLAS/LAPACK vinculado
5 print("Configuración de NumPy y BLAS:")
6 np.show_config()
```

1.2. Organización en Memoria: Disposición C vs. Fortran

Un arreglo bidimensional puede almacenarse por filas (*C-contiguous*) o por columnas (*Fortran-contiguous*). Comprender esto es vital para maximizar los aciertos en la memoria caché (*cache hits*):

```
1 A_c = np.array([[1.0, 2.0], [3.0, 4.0]], order='C')
2 A_f = np.array([[1.0, 2.0], [3.0, 4.0]], order='F')
3
4 print("Es C-contiguo:", A_c.flags['C_CONTIGUOUS']) # True
5 print("Es Fortran-contiguo:", A_f.flags['F_CONTIGUOUS']) # True
```

1.3. Reglas de Difusión (*Broadcasting*)

El *broadcasting* describe cómo NumPy gestiona arreglos de formas diferentes durante operaciones aritméticas elemento por elemento, sin duplicar memoria explícitamente:

1. Si dos arreglos tienen un número distinto de dimensiones, se añaden dimensiones de tamaño 1 a la izquierda del de menor dimensión.
2. Dos dimensiones son compatibles si son iguales o si una de ellas es 1.
3. Si una dimensión es 1, el arreglo actúa como si sus datos se hubieran duplicado a lo largo de esa dimensión.

```

1 # Matriz 3x2 sumada a vector fila 1x2 -> broadcasting automatico a 3x2
2 X = np.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]]) # shape (3, 2)
3 mu = np.array([1.0, 2.0]) # shape (2,)
4
5 # Centrado de datos: cada fila resta la media correspondiente
6 X_centrado = X - mu # Sin necesidad de bucle for
7 print("Matriz centrada:\n", X_centrado)

```

▲ [Regresar al Índice General](#)

2. Vectores y Espacios Vectoriales

2.1. Definición Matemática y Representación Computacional

Un vector columna $\mathbf{v} \in \mathbb{R}^n$ es una tupla ordenada de n números reales:

$$\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix} \quad (1)$$

En NumPy, los arreglos unidimensionales (`shape: (n,)`) representan vectores genéricos:

```

1 u = np.array([1.0, 2.0, 3.0])
2 v = np.array([4.0, 0.0, -1.0])
3
4 # Suma y multiplicacion por escalar
5 w = 2.5 * u - 1.2 * v
6 print("Combinacion lineal 2.5*u - 1.2*v:", w)

```

2.2. Independencia Lineal y Combinación Lineal

Un conjunto de vectores $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ es **linealmente independiente** si la única solución a la ecuación:

$$c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_k \mathbf{v}_k = \mathbf{0} \quad (2)$$

es la solución trivial $c_1 = c_2 = \dots = c_k = 0$.

Criterio Computacional de Independencia Lineal

Formando la matriz $V = [\mathbf{v}_1 \mid \dots \mid \mathbf{v}_k]$, el conjunto es linealmente independiente si y solo si:

$$\text{rango}(V) = k$$

```

1 # Verificacion de independencia lineal para 3 vectores en R^3
2 v1 = np.array([1.0, 2.0, 3.0])
3 v2 = np.array([0.0, 1.0, 4.0])
4 v3 = np.array([5.0, 6.0, 0.0])
5
6 V = np.column_stack([v1, v2, v3])
7 rango = np.linalg.matrix_rank(V)
8 es_independiente = (rango == V.shape[1])
9 print(f"Rango de V: {rango}. Es independiente: {es_independiente}")

```

2.3. Producto Escalar (Producto Punto)

El producto interno euclidiano entre $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ se define como:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i v_i \quad (3)$$

Propiedades clave:

- Conmutatividad: $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$.
- Linealidad: $(a\mathbf{u} + \mathbf{w}) \cdot \mathbf{v} = a(\mathbf{u} \cdot \mathbf{v}) + (\mathbf{w} \cdot \mathbf{v})$.
- Positividad: $\mathbf{u} \cdot \mathbf{u} \geq 0$, con igualdad si y solo si $\mathbf{u} = \mathbf{0}$.

```
1 # Sintaxis moderna: operador infijo @
2 dot_val = u @ v
3 print("Producto punto u @ v:", dot_val)
```

2.4. Familias de Normas Vectoriales (L_1 , L_2 , L_∞)

Una norma $\|\cdot\|$ asigna una longitud métrica al vector. La norma general L_p se formula como:

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}, \quad p \geq 1 \quad (4)$$

- Norma L_1 (Manhattan): $\|\mathbf{v}\|_1 = \sum |v_i|$ (promueve dispersión / *sparsity* en regularización Lasso).
- Norma L_2 (Euclidiana): $\|\mathbf{v}\|_2 = \sqrt{\sum v_i^2} = \sqrt{\mathbf{v} \cdot \mathbf{v}}$ (geometría clásica y regularización Ridge).
- Norma L_∞ (Chebyshev): $\|\mathbf{v}\|_\infty = \max_i |v_i|$.

```
1 x = np.array([3.0, -4.0, 5.0])
2
3 norm_1 = np.linalg.norm(x, ord=1) # 3 + 4 + 5 = 12
4 norm_2 = np.linalg.norm(x, ord=2) # sqrt(9 + 16 + 25) = sqrt(50) ~ 7.071
5 norm_inf = np.linalg.norm(x, ord=np.inf) # max(3, 4, 5) = 5
6
7 print(f"Norma L1: {norm_1}, Norma L2: {norm_2:.4f}, Norma L_inf: {norm_inf}")
```

2.5. Distancia Euclidiana y Similitud del Coseno

La distancia euclidiana es $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2$.

En procesamiento de lenguaje natural (NLP) y sistemas de recomendación, la métrica predominante es la **similitud del coseno**, que cuantifica la alineación angular sin verse afectada por la magnitud absoluta:

$$S_C(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \cos(\theta) \in [-1, 1] \quad (5)$$

```
1 sim_cos = (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))
2 theta_deg = np.degrees(np.arccos(np.clip(sim_cos, -1.0, 1.0)))
3 print(f"Similitud del coseno: {sim_cos:.4f}, Angulo: {theta_deg:.2f} grados")
```

2.6. Proyección Ortogonal de un Vector

La proyección ortogonal del vector $\mathbf{v}$ sobre la dirección del vector $\mathbf{u} \neq \mathbf{0}$ descompone a $\mathbf{v}$ en una componente paralela $\mathbf{v}_{\parallel}$ y una ortogonal $\mathbf{v}_{\perp}$:

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2^2} \mathbf{u}, \quad \mathbf{v}_{\perp} = \mathbf{v} - \text{proj}_{\mathbf{u}}(\mathbf{v}) \quad (6)$$

```
1 proj_u_v = ((u @ v) / (np.linalg.norm(u)**2)) * u
2 v_perp = v - proj_u_v
3
4 print("Proyeccion de v sobre u:", proj_u_v)
5 print("Componente ortogonal:", v_perp)
6 print("Verificacion ortogonalidad proj . perp:", np.isclose(proj_u_v @
    v_perp, 0.0))
```

2.7. Producto Cruz en $\mathbb{R}^3$

El producto cruz produce un vector perpendicular al plano generado por $\mathbf{u}$ y $\mathbf{v}$, cuya norma equivale al área del paralelogramo delimitado por ambos:

$$\mathbf{u} \times \mathbf{v} = \det \begin{pmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{pmatrix} = \begin{pmatrix} u_2 v_3 - u_3 v_2 \\ u_3 v_1 - u_1 v_3 \\ u_1 v_2 - u_2 v_1 \end{pmatrix} \quad (7)$$

```
1 cross_uv = np.cross(u, v)
2 area_paralelogramo = np.linalg.norm(cross_uv)
3 print("Producto vectorial u x v:", cross_uv)
4 print("Area del paralelogramo:", area_paralelogramo)
```

[▲ Regresar al Índice General](#)

3. Matrices, Álgebra Matricial y Operaciones Tensoriales

3.1. Tipología y Familias de Matrices

Una matriz $A \in \mathbb{R}^{m \times n}$ opera como un mapeo lineal entre espacios vectoriales. Distinguimos clases estructurales esenciales:

- **Diagonal:** $A_{ij} = 0$ para todo $i \neq j$.
- **Simétrica:** $A = A^T$ (posee autovalores reales y autovectores ortonormales).
- **Antisimétrica:** $A = -A^T$ (diagonal nula, autovalores puramente imaginarios).
- **Ortogonal:** $Q^T Q = Q Q^T = I$ (preserva longitudes y ángulos euclidianos, $\|Q\mathbf{x}\| = \|\mathbf{x}\|$).

```
1 # Matrices estructuradas en NumPy
2 I3 = np.eye(3) # Identidad 3x3
3 D = np.diag([4.0, -1.0, 7.0]) # Diagonal
4 S = np.array([[2.0, 1.0], [1.0, 5.0]]) # Simetrica (S == S.T)
5 print("Es simetrica S:", np.allclose(S, S.T))
```

3.2. Multiplicación Matricial, Producto Exterior y de Kronecker

Distinción Crucial: *, @, np.outer y np.kron

- **Elemento a elemento (Hadamard):** $A * B \rightarrow (A \odot B)_{ij} = A_{ij}B_{ij}$.
- **Matricial estándar:** $A @ B \rightarrow (AB)_{ij} = \sum_k A_{ik}B_{kj}$.
- **Producto exterior:** $\text{np.outer}(u, v) \rightarrow uv^T \in \mathbb{R}^{m \times n}$ (matriz de rango 1).
- **Producto de Kronecker:** $\text{np.kron}(A, B)$ (matriz por bloques para computación cuántica y análisis multivariado).

```

1 A = np.array([[1.0, 2.0], [3.0, 4.0]])
2 B = np.array([[2.0, 0.0], [1.0, 3.0]])
3
4 print("Hadamard (A * B):\n", A * B)
5 print("Multiplicacion Algebraica (A @ B):\n", A @ B)
6 print("Producto exterior u (x) v:\n", np.outer(np.array([1, 2]), np.array([3,
    4])))

```

3.3. Traza y sus Propiedades Cíclicas

La traza es la suma de los elementos diagonales: $\text{tr}(A) = \sum_{i=1}^n a_{ii}$. Propiedad fundamental en estadística y teoría espectral:

$$\text{tr}(ABC) = \text{tr}(CAB) = \text{tr}(BCA), \quad \text{tr}(A) = \sum_{i=1}^n \lambda_i \quad (8)$$

```

1 M = np.array([[5.0, 2.0], [3.0, 8.0]])
2 print("Traza de M:", np.trace(M)) # 5 + 8 = 13.0

```

3.4. Notación de Suma de Einstein con np.einsum

La función `np.einsum` permite especificar contracciones, productos tensoriales y trazas con máxima claridad y velocidad computacional extrema:

```

1 # 1. Traza de una matriz: sum_i A_ii
2 tr_ein = np.einsum('ii->', M)
3
4 # 2. Multiplicacion matricial: C_ik = sum_j A_ij * B_jk
5 mul_ein = np.einsum('ij,jk->ik', A, B)
6
7 # 3. Suma a lo largo de columnas: sum_i A_ij
8 col_sum = np.einsum('ij->j', A)
9
10 print("Traza con einsum:", tr_ein)
11 print("Multiplicacion con einsum valida:", np.allclose(mul_ein, A @ B))

```

[▲ Regresar al Índice General](#)

4. Transformaciones Lineales y Geometría en 2D y 3D

4.1. Definición de Operador Lineal

Una función $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ es una transformación lineal si preserva las combinaciones lineales:

$$T(c\mathbf{u} + \mathbf{v}) = cT(\mathbf{u}) + T(\mathbf{v}), \quad \forall c \in \mathbb{R}, \mathbf{u}, \mathbf{v} \in \mathbb{R}^n \quad (9)$$

Toda transformación lineal finita queda unívocamente determinada por su acción sobre los vectores de la base canónica $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$, condensándose en una matriz de transformación A :

$$T(\mathbf{x}) = A\mathbf{x} \quad (10)$$

4.2. Matrices de Transformación Clásicas en 2D

1. Rotación en 2D en sentido antihorario por un ángulo θ :

$$R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \quad (11)$$

2. Escalado con factores (s_x, s_y) :

$$S = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \quad (12)$$

3. Cizallamiento (*Shear*) horizontal y vertical:

$$K_x = \begin{pmatrix} 1 & k_x \\ 0 & 1 \end{pmatrix}, \quad K_y = \begin{pmatrix} 1 & 0 \\ k_y & 1 \end{pmatrix} \quad (13)$$

4. Reflexión respecto al eje X :

$$F_x = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (14)$$

```

1  # Implementacion de operadores geometricos 2D
2  def matriz_rotacion_2d(grados):
3      theta = np.radians(grados)
4      c, s = np.cos(theta), np.sin(theta)
5      return np.array([[c, -s], [s, c]])
6
7  # Malla de puntos de un cuadrado unitario
8  puntos = np.array([
9      [0.0, 0.0],
10     [1.0, 0.0],
11     [1.0, 1.0],
12     [0.0, 1.0]
13 ]).T # shape: (2, 4)
14
15 # Aplicar rotacion de 45 grados seguida de escalado x2 en Y
16 R_45 = matriz_rotacion_2d(45.0)
17 S = np.diag([1.0, 2.0])
18
19 # Composicion matricial: primero rotar, luego escalar
20 M_compuesta = S @ R_45
21 puntos_transformados = M_compuesta @ puntos
22
23 print("Puntos originales:\n", puntos)
24 print("Puntos transformados:\n", np.round(puntos_transformados, 3))

```

▲ [Regresar al Índice General](#)

5. Determinante, Rango y los Cuatro Subespacios Fundamentales

5.1. Determinante e Interpretación Geométrica

El determinante $\det(A)$ cuantifica la variación del volumen orientado inducida por la transformación A .

- En 2D: $\det(A)$ representa el área orientada del paralelogramo delimitado por las columnas.
- En 3D: $\det(A)$ representa el volumen orientado del paralelepípedo.
- Si $\det(A) = 0$, la transformación comprime el espacio a una dimensión inferior; la matriz es **singular** y no posee inversa clásica.

Estabilidad Numérica con Determinantes: `np.linalg.slogdet`

En dimensiones altas ($n > 20$), el determinante computado directamente con `np.linalg.det` puede sufrir desbordamiento (*overflow*) o subdesbordamiento (*underflow*). La forma robusta y recomendada en computación científica es calcular el signo y el logaritmo del determinante mediante `np.linalg.slogdet`:

$$\det(A) = \text{sign} \cdot \exp(\log\det)$$

```
1 A_grande = np.random.randn(50, 50)
2 signo, logdet = np.linalg.slogdet(A_grande)
3 print(f"Signo del determinante: {signo}, Log-determinante: {logdet:.4f}")
```

5.2. Los Cuatro Subespacios Fundamentales de Strang

Dada una matriz $A \in \mathbb{R}^{m \times n}$ con rango r :

1. **Espacio Columna / Imagen** $\mathcal{C}(A) \subseteq \mathbb{R}^m$: Dimensión r .
2. **Espacio Nulo / Kernel** $\mathcal{N}(A) \subseteq \mathbb{R}^n$: Soluciones a $A\mathbf{x} = \mathbf{0}$. Dimensión $n - r$.
3. **Espacio Fila** $\mathcal{C}(A^T) \subseteq \mathbb{R}^n$: Dimensión r . Complemento ortogonal a $\mathcal{N}(A)$.
4. **Espacio Nulo Izquierdo** $\mathcal{N}(A^T) \subseteq \mathbb{R}^m$: Soluciones a $A^T\mathbf{y} = \mathbf{0}$. Dimensión $m - r$.

Teorema del Rango-Nulidad

Para toda matriz $A \in \mathbb{R}^{m \times n}$:

$$\text{rango}(A) + \text{nulidad}(A) = n$$

donde $\text{nulidad}(A) = \dim(\mathcal{N}(A))$.

```
1 import scipy.linalg as la
2
3 # Matriz 3x4 de rango 2
4 A = np.array([
5     [1.0, 2.0, 0.0, 3.0],
6     [2.0, 4.0, 1.0, 7.0],
7     [1.0, 2.0, 1.0, 4.0]
8 ])
9
10 r = np.linalg.matrix_rank(A)
```

```

11 n = A.shape[1]
12
13 # Calculo de base ortonormal para el espacio nulo (Kernel)
14 null_A = la.null_space(A)
15
16 print(f"Rango: {r}, Nulidad teorica (n - r): {n - r}")
17 print("Dimension del espacio nulo obtenido:", null_A.shape[1])
18 print("Base ortonormal del espacio nulo:\n", np.round(null_A, 4))
19 # Comprobacion: A @ null_A == 0
20 print("Comprobacion A @ Kernel == 0:", np.allclose(A @ null_A, 0.0))

```

5.3. Inversa Clásica y Pseudoinversa de Moore-Penrose

Si $A \in \mathbb{R}^{n \times n}$ es no singular ($\det(A) \neq 0$), existe una única matriz A^{-1} tal que $AA^{-1} = A^{-1}A = I_n$.

Para matrices no cuadradas ($m \neq n$) o matrices singulares, la **pseudoinversa de Moore-Penrose** $A^+ \in \mathbb{R}^{n \times m}$ proporciona una inversa generalizada basada en SVD:

$$A^+ = V\Sigma^+U^T \quad (15)$$

```

1 # Matriz rectangular 3x2
2 A_rect = np.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
3 A_pinv = np.linalg.pinv(A_rect)
4
5 print("Pseudoinversa A^+:\n", np.round(A_pinv, 4))
6 # Propiedad fundamental: A @ A^+ @ A == A
7 print("Verificacion A @ A^+ @ A == A:", np.allclose(A_rect @ A_pinv @ A_rect,
    A_rect))

```

[▲ Regresar al Índice General](#)

6. Sistemas de Ecuaciones Lineales y Análisis Numérico

6.1. Formulación Matricial y Teorema de Rouché-Frobenius

Un sistema de m ecuaciones lineales con n incógnitas adopta la forma canónica:

$$Ax = b, \quad A \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m \quad (16)$$

Sea $(A \mid b)$ la matriz ampliada:

- **Incompatible (sin solución):** $\text{rango}(A) < \text{rango}(A \mid b)$.
- **Compatible Determinado (solución única):** $\text{rango}(A) = \text{rango}(A \mid b) = n$.
- **Compatible Indeterminado (infinitas soluciones):** $\text{rango}(A) = \text{rango}(A \mid b) < n$.

Regla de Oro Numérica: NUNCA usar `inv(A) @ b`

El cálculo explícito de la inversa A^{-1} es altamente inestable frente a errores de redondeo de punto flotante de 64 bits y requiere aproximadamente el triple de operaciones que la descomposición LU:

- **Incorrecto e Inestable:** `x = np.linalg.inv(A) @ b`

- **Correcto, Óptimo y Robusto:** $\mathbf{x} = \text{np.linalg.solve}(\mathbf{A}, \mathbf{b})$

```

1 A_sys = np.array([
2     [ 2.0,  1.0,  1.0],
3     [-3.0, -1.0,  2.0],
4     [-2.0,  1.0,  2.0]
5 ])
6 b_sys = np.array([8.0, -11.0, -3.0])
7
8 # Resolución óptima mediante factorización LU con pivoteo parcial
9 x_sol = np.linalg.solve(A_sys, b_sys)
10 residuo = np.linalg.norm(A_sys @ x_sol - b_sys)
11
12 print("Solucion analitica x:", x_sol)
13 print(f"Norma del residuo ||Ax - b||: {residuo:.2e}")

```

6.2. Número de Condición y Estabilidad Numérica

El **número de condición** $\kappa(A)$ cuantifica la sensibilidad de la solución $\mathbf{x}$ ante pequeñas perturbaciones en los coeficientes de A o en el vector $\mathbf{b}$:

$$\kappa(A) = \|A\|_2 \cdot \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \geq 1 \quad (17)$$

- $\kappa(A) \approx 1$: Matriz **bien condicionada** (errores mínimos).
- $\kappa(A) \gg 10^7$: Matriz **mal condicionada** (se pierden aproximadamente $\log_{10} \kappa(A)$ dígitos decimales de precisión en aritmética float64).

```

1 # Matriz de Hilbert 5x5 (clásicamente mal condicionada)
2 H = la.hilbert(5)
3 cond_H = np.linalg.cond(H)
4
5 print(f"Numero de condicion de Hilbert 5x5: {cond_H:.2e}")
6 # Una pequeña perturbacion en b provocara fluctuaciones gigantescas en x

```

6.3. Sistemas Sobredeterminados y Mínimos Cuadrados (OLS)

Cuando $m > n$, el sistema carece de solución exacta. El método de mínimos cuadrados busca el vector $\mathbf{x}^*$ que minimiza la suma de errores cuadráticos:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2^2 \quad (18)$$

Geométricamente, $\mathbf{A}\mathbf{x}^*$ es la proyección ortogonal de $\mathbf{b}$ sobre $\mathcal{C}(A)$. Las **ecuaciones normales** son:

$$\mathbf{A}^T \mathbf{A} \mathbf{x}^* = \mathbf{A}^T \mathbf{b} \implies \mathbf{x}^* = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b} = \mathbf{A}^+ \mathbf{b} \quad (19)$$

```

1 # Ajuste lineal y = m*x + c con 5 observaciones
2 x_data = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
3 y_data = np.array([2.1, 3.9, 6.1, 8.2, 9.8])
4
5 # Matriz de diseño con columna de unos para el sesgo (intercepto)
6 A_ols = np.column_stack([x_data, np.ones_like(x_data)])
7
8 # Resolución mediante SVD (máxima estabilidad numérica)
9 theta, residuals, rank, s = np.linalg.lstsq(A_ols, y_data, rcond=None)

```

```

10 pendiente, intercepto = theta
11
12 print(f"Modelo ajustado: y = {pendiente:.4f} * x + {intercepto:.4f}")

```

[▲ Regresar al Índice General](#)

7. Teoría Espectral: Autovalores, Autovectores y Diagonalización

7.1. Definición y Polinomio Característico

Para una matriz cuadrada $A \in \mathbb{R}^{n \times n}$, un escalar $\lambda \in \mathbb{C}$ y un vector no nulo $\mathbf{v} \in \mathbb{C}^n$ son respectivamente un **autovalor** (*eigenvalue*) y un **autovector** (*eigenvector*) si:

$$A\mathbf{v} = \lambda\mathbf{v} \iff (A - \lambda I_n)\mathbf{v} = \mathbf{0} \quad (20)$$

Para la existencia de soluciones no triviales se debe cumplir la **ecuación característica**:

$$p(\lambda) = \det(A - \lambda I_n) = 0 \quad (21)$$

7.2. Diagonalización de Matrices

Si A tiene n autovectores linealmente independientes agrupados como columnas en una matriz $P = [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_n]$, y $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$:

$$A = P\Lambda P^{-1} \implies A^k = P\Lambda^k P^{-1} \quad (22)$$

Esto permite calcular potencias de matrices y exponenciales matriciales e^{At} a coste $O(n)$ tras la diagonalización inicial.

```

1 A_mat = np.array([
2     [4.0, -2.0],
3     [1.0,  1.0]
4 ])
5
6 eigenvalores, P = np.linalg.eig(A_mat)
7 Lambda = np.diag(eigenvalores)
8
9 print("Autovalores:", eigenvalores)
10 print("Matriz modal P:\n", P)
11
12 # Reconstruccion: A == P @ Lambda @ inv(P)
13 A_reconst = P @ Lambda @ np.linalg.inv(P)
14 print("Reconstruccion valida:", np.allclose(A_mat, A_reconst))
15
16 # Potencia A^10 calculada via diagonalizacion
17 A_10 = P @ np.diag(eigenvalores**10) @ np.linalg.inv(P)
18 print("A^10 [0,0]:", A_10[0, 0])

```

7.3. El Teorema Espectral para Matrices Simétricas

Teorema Espectral Real

Si $A \in \mathbb{R}^{n \times n}$ es simétrica ($A = A^T$):

1. Todos sus autovalores λ_i son estrictamente reales ($\lambda_i \in \mathbb{R}$).
2. Los autovectores correspondientes a autovalores distintos son mutuamente ortogonales.
3. Existe una base ortonormal de autovectores que conforma una matriz ortogonal Q ($Q^T Q = I$):

$$A = Q \Lambda Q^T = \sum_{i=1}^n \lambda_i \mathbf{q}_i \mathbf{q}_i^T$$

Uso de `np.linalg.eigh`

Para cualquier matriz simétrica o hermitiana, use **siempre** `np.linalg.eigh` en lugar de `np.linalg.eig`. Garantiza autovalores puramente reales y ortonormalidad estricta con algoritmos divide-and-conquer mucho más rápidos.

```
1 S_mat = np.array([
2     [3.0, 1.0, 0.0],
3     [1.0, 4.0, 1.0],
4     [0.0, 1.0, 3.0]
5 ])
6
7 evals, Q = np.linalg.eigh(S_mat)
8 print("Autovalores reales ordenados:", evals)
9 print("Matriz Q ortogonal (Q @ Q.T == I):", np.allclose(Q @ Q.T, np.eye(3)))
```

7.4. Matrices Definidas Positivas

Una matriz simétrica A es **definida positiva** ($A \succ 0$) si $\mathbf{x}^T A \mathbf{x} > 0$ para todo $\mathbf{x} \neq \mathbf{0}$. Esto equivale a que todos sus autovalores sean estrictamente positivos:

$$\lambda_i(A) > 0, \quad \forall i = 1, \dots, n \quad (23)$$

Propiedad indispensable en optimización convexa (la matriz Hessiana define mínimos locales estrictos).

[▲ Regresar al Índice General](#)

8. Descomposiciones Matriciales Fundamentales

Las factorizaciones transforman una matriz compleja en el producto de matrices estructuradas (triangulares, diagonales u ortogonales), reduciendo drásticamente el coste y la inestabilidad de los algoritmos.

8.1. Descomposición LU con Pivoteo Parcial ($PA = LU$)

Factoriza cualquier matriz cuadrada A en el producto de una matriz de permutación P , una triangular inferior L (con unos en la diagonal) y una triangular superior U :

$$PA = LU \quad (24)$$

Es la base interna de `np.linalg.solve`: resolver $Ax = b$ equivale a resolver dos sistemas triangulares por sustitución progresiva y regresiva en tiempo $O(n^2)$.

```

1 A_lu = np.array([
2     [2.0, 1.0, 1.0],
3     [4.0, 3.0, 3.0],
4     [8.0, 7.0, 9.0]
5 ])
6
7 P_mat, L_mat, U_mat = la.lu(A_lu)
8 print("Matriz P (permutacion):\n", P_mat)
9 print("Matriz L (triangular inferior):\n", np.round(L_mat, 3))
10 print("Matriz U (triangular superior):\n", np.round(U_mat, 3))
11 print("Comprobacion P @ A == L @ U:", np.allclose(P_mat @ A_lu, L_mat @
    U_mat))

```

8.2. Descomposición de Cholesky ($A = LL^T$)

Exclusiva para matrices simétricas y definidas positivas ($A \succ 0$). Factoriza la matriz en una única matriz triangular inferior L :

$$A = LL^T \quad (25)$$

Es dos veces más rápida que la factorización LU y posee una estabilidad numérica incondicional. Se utiliza ampliamente en Procesos Gaussianos, Filtros de Kalman y simulaciones Monte Carlo de variables correlacionadas.

```

1 # Construccion de una matriz simetrica definida positiva: A = M @ M.T
2 M_aux = np.array([[2.0, 1.0], [-1.0, 3.0]])
3 A_spd = M_aux @ M_aux.T
4
5 L_chol = np.linalg.cholesky(A_spd)
6 print("Factor de Cholesky L:\n", L_chol)
7 print("Verificacion L @ L.T == A:", np.allclose(L_chol @ L_chol.T, A_spd))

```

8.3. Descomposición QR ($A = QR$)

Factoriza cualquier matriz $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) en una matriz con columnas ortonormales $Q \in \mathbb{R}^{m \times n}$ ($Q^T Q = I_n$) y una triangular superior $R \in \mathbb{R}^{n \times n}$:

$$A = QR \quad (26)$$

Es el método canónico para resolver mínimos cuadrados y el núcleo del algoritmo iterativo para calcular autovalores.

```

1 A_qr = np.array([
2     [1.0, -1.0],
3     [1.0, 0.0],
4     [1.0, 1.0]
5 ])
6
7 Q, R = np.linalg.qr(A_qr)
8 print("Matriz Q (columnas ortonormales):\n", np.round(Q, 4))
9 print("Matriz R (triangular superior):\n", np.round(R, 4))
10 print("Comprobacion Q.T @ Q == I:\n", np.round(Q.T @ Q, 6))

```

8.4. Descomposición en Valores Singulares (SVD)

La SVD es considerada el teorema culminante del álgebra lineal, aplicable a cualquier matriz arbitraria rectangular $A \in \mathbb{R}^{m \times n}$:

$$A = U \Sigma V^T = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (27)$$

donde:

- $U \in \mathbb{R}^{m \times m}$ es una matriz ortogonal con los autovectores de AA^T (*vectores singulares izquierdos*).
- $V \in \mathbb{R}^{n \times n}$ es una matriz ortogonal con los autovectores de $A^T A$ (*vectores singulares derechos*).
- $\Sigma \in \mathbb{R}^{m \times n}$ es una matriz diagonal con los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, donde $\sigma_i = \sqrt{\lambda_i(A^T A)}$.

Teorema de Eckart-Young-Mirsky (Aproximación de Bajo Rango)

La mejor aproximación de rango $k < r$ de una matriz A bajo la norma de Frobenius o la norma espectral se obtiene truncando la suma SVD en los primeros k términos:

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

```

1 # Ejemplo de aproximacion de bajo rango via SVD
2 np.random.seed(42)
3 A_datos = np.random.randn(20, 10)
4
5 U, s, Vt = np.linalg.svd(A_datos, full_matrices=False)
6 print("Valores singulares s (orden decreciente):\n", np.round(s, 3))
7
8 # Reconstruccion con rango reducido k = 3
9 k = 3
10 A_k = U[:, :k] @ np.diag(s[:k]) @ Vt[:, :k]
11 error_frobenius = np.linalg.norm(A_datos - A_k, 'fro')
12 print(f"Error de aproximacion con rango k={k}: {error_frobenius:.4f}")

```

▲ [Regresar al Índice General](#)

9. Aplicaciones en Machine Learning y Ciencia de Datos

9.1. Análisis de Componentes Principales (PCA) desde Cero

El PCA busca proyecciones lineales que maximicen la varianza de los datos o minimicen el error cuadrático medio de reconstrucción. Se reduce al análisis espectral de la matriz de covarianza muestral $\Sigma_X = \frac{1}{n-1} X^T X$:

Algoritmo Matemático de PCA

1. Centrar la matriz de diseño: $\tilde{X} = X - \mu$.
2. Calcular la descomposición SVD directa de $\tilde{X}$:

$$\tilde{X} = U\Sigma V^T$$

3. Las columnas de V son las **direcciones principales** (autovectores de la covarianza).
4. Los datos proyectados en el espacio latente de dimensión k son: $Z = \tilde{X}V_k = U_k\Sigma_k$.

```

1 def pca_desde_cero(X, n_componentes=2):
2     # 1. Centrado
3     media = np.mean(X, axis=0)
4     X_centrado = X - media
5
6     # 2. SVD directa (computacionalmente mas estable que calcular X.T @ X)
7     U, s, Vt = np.linalg.svd(X_centrado, full_matrices=False)
8
9     # 3. Componentes principales y proyeccion
10    componentes = Vt[:n_componentes, :].T
11    proyeccion = X_centrado @ componentes
12
13    # Varianza explicada
14    varianza_explicada = (s**2) / (X.shape[0] - 1)
15    ratio_varianza = varianza_explicada[:n_componentes] /
16        np.sum(varianza_explicada)
17
18    return proyeccion, componentes, ratio_varianza
19
20 # Prueba sintetica con 100 muestras y 5 caracteristicas
21 X_dummy = np.random.randn(100, 5) @ np.random.randn(5, 5)
22 Z, comps, ratios = pca_desde_cero(X_dummy, n_componentes=2)
23
24 print(f"Dimension reducida: {Z.shape}")
25 print(f"Ratios de varianza explicada por los 2 primeros ejes: {ratios}")

```

9.2. Algoritmo PageRank: Autovectores en Grafos Web

El algoritmo fundacional de Google modela la navegación web como una cadena de Markov ergódica. Si M es la matriz estocástica de transición y $\alpha \approx 0.85$ es el factor de amortiguamiento (*damping factor*):

$$G = \alpha M + \frac{1 - \alpha}{n} \mathbf{E} \quad (28)$$

El vector de puntuaciones de PageRank $\mathbf{p}^*$ es el **autovector estacionario** asociado al autovalor $\lambda = 1$:

$$G\mathbf{p}^* = \mathbf{p}^*, \quad \sum_{i=1}^n p_i^* = 1 \quad (29)$$

```

1 def calcular_pagerank(M, alpha=0.85, max_iter=100, tol=1e-8):
2     n = M.shape[0]
3     p = np.ones(n) / n # Distribucion uniforme inicial
4     G = alpha * M + (1.0 - alpha) / n * np.ones((n, n))
5
6     # Metodo de las potencias iterativo
7     for _ in range(max_iter):
8         p_siguiente = G @ p
9         if np.linalg.norm(p_siguiente - p, 1) < tol:

```



```
10         break
11         p = p_siguiete
12
13     return p / np.sum(p)
14
15 # Grafo web de juguete con 4 paginas
16 M_web = np.array([
17     [0.0, 0.5, 0.0, 0.0],
18     [0.33, 0.0, 0.0, 1.0],
19     [0.33, 0.5, 0.0, 0.0],
20     [0.33, 0.0, 1.0, 0.0]
21 ])
22
23 scores = calcular_pagerank(M_web)
24 print("Scores de PageRank por pagina:", np.round(scores, 4))
```

[▲ Regresar al Índice General](#)

10. Tabla Resumen de Operaciones y Complejidad Numérica

La siguiente tabla compendia la correspondencia formal entre la notación matemática rigurosa, la función óptima de NumPy / SciPy y la complejidad asintótica del algoritmo:

Concepto	Símbolo	NumPy / SciPy	Coste	Observaciones
Vector / Matriz	$\mathbf{v}, A$	<code>np.array([...])</code>	$O(1)$	Bloque de memoria contiguo
Producto Escalar	$\mathbf{u} \cdot \mathbf{v}$	<code>u @ v</code>	$O(n)$	Vectorizado con SIMD / BLAS
Norma L_2	$\ \mathbf{v}\ _2$	<code>np.linalg.norm(v)</code>	$O(n)$	Evita overflow interno
Producto Cruz	$\mathbf{u} \times \mathbf{v}$	<code>np.cross(u, v)</code>	$O(1)$	Restringido a $\mathbb{R}^3$
Transpuesta	A^T	<code>A.T</code>	$O(1)$	Vista en memoria (<i>strides</i>)
Mult. Matricial	AB	<code>A @ B</code>	$O(n^3)$	Rutina <code>dgemm</code> de BLAS
Hadamard	$A \odot B$	<code>A * B</code>	$O(mn)$	Elemento a elemento
Contracción	Tensorial	<code>np.einsum('ij,jk->ik', A, B)</code>	Var.	Notación Einstein óptima
Determinante	$\det(A)$	<code>np.linalg.slogdet(A)</code>	$O(n^3)$	Usa signo y logaritmo
Traza	$\text{tr}(A)$	<code>np.trace(A)</code>	$O(n)$	Suma elementos diagonales
Rango	$\text{rg}(A)$	<code>np.linalg.matrix_rank(A)</code>	$O(mn^2)$	Basado en truncamiento SVD
Espacio Nulo	$\mathcal{N}(A)$	<code>scipy.linalg.null_space(A)</code>	$O(mn^2)$	Base ortonormal del Kernel
Inversa	A^{-1}	<code>np.linalg.inv(A)</code>	$O(n^3)$	No usar para resolver $A\mathbf{x} = \mathbf{b}$
Pseudoinversa	A^+	<code>np.linalg.pinv(A)</code>	$O(mn^2)$	Moore-Penrose vía SVD
Sistema Lineal	$A\mathbf{x} = \mathbf{b}$	<code>np.linalg.solve(A, b)</code>	$O(n^3/3)$	Factorización LU con pivoteo
Mínimos Cuad.	$\min \ b - Ax\ $	<code>np.linalg.lstsq(A, b)</code>	$O(mn^2)$	Descomposición SVD
Factorización LU	$PA = LU$	<code>scipy.linalg.lu(A)</code>	$O(n^3/3)$	Pivoteo parcial de filas
Cholesky	$A = LL^T$	<code>np.linalg.cholesky(A)</code>	$O(n^3/6)$	Exclusivo para $A \succ 0$
Factorización QR	$A = QR$	<code>np.linalg.qr(A)</code>	$O(mn^2)$	Reflexiones de Householder
Autovalores	$A\mathbf{v} = \lambda\mathbf{v}$	<code>np.linalg.eig(A)</code>	$O(n^3)$	Algoritmo QR iterativo
Autoval. Simétricos	$S = Q\Lambda Q^T$	<code>np.linalg.eigh(S)</code>	$O(n^3)$	Precisión y velocidad superior
SVD	$U\Sigma V^T$	<code>np.linalg.svd(A)</code>	$O(mn^2)$	Factorización general universal

Tabla 1: Mapeo riguroso de operaciones matriciales, sintaxis computacional y complejidades asintóticas.

[▲ Regresar al Índice General](#)

11. Decálogo de Buenas Prácticas Computacionales y Conclusiones

Para maximizar el rendimiento, la exactitud matemática y la estabilidad de las implementaciones en Python:

1. **Elimine bucles for nativos:** Reemplace cualquier iteración manual sobre filas o columnas por expresiones vectorizadas o *broadcasting*. Las rutinas de NumPy se ejecutan en código nativo compilado con registros SIMD.
2. **Prohibido invertir matrices para resolver sistemas:** Nunca ejecute `np.linalg.inv(A) @ b`. Utilice `np.linalg.solve(A, b)`, el cual reduce el error de redondeo y triplica la velocidad de ejecución.
3. **Monitoree el número de condición:** Evalúe siempre `np.linalg.cond(A)`. Si $\kappa(A) \geq 10^{10}$, regularice el problema (añadiendo una pequeña diagonal de Tikhonov λI) para evitar soluciones numéricas erróneas.
4. **Priorice `eigh` sobre `eig` en matrices simétricas:** Si la matriz proviene de una covarianza, un grafo no dirigido o un operador Laplaciano, `eigh` asegura autovalores reales y ortogonalidad exacta.
5. **Vigile la precisión del tipo de dato (`dtype`):** Use `float64` por defecto para cómputo científico de alta precisión. En modelos de Deep Learning con aceleración por GPU, evalúe `float32` o `bfloat16` para duplicar el rendimiento y reducir el consumo de memoria a la mitad.
6. **Preasigne memoria contigua:** Evite concatenaciones sucesivas dentro de bucles (`np.append` o `np.vstack`), ya que recrean copias completas en memoria $O(N^2)$. Preasigne con `np.empty` o `np.zeros`.
7. **Verifique tolerancias numéricas con `np.isclose` o `np.allclose`:** En aritmética de coma flotante, la comparación estricta `x == y` y casi siempre falla debido al redondeo IEEE-754.
8. **Aproveche `scipy.linalg.null_space`:** Cuando trabaje con sistemas subdeterminados o busque el núcleo de transformaciones singulares, obtenga bases ortonormales numéricamente exactas.
9. **Implemente PCA mediante SVD en lugar de la covarianza explícita:** Computar $X^T X$ eleva al cuadrado el número de condición ($\kappa(X^T X) = \kappa(X)^2$). Aplicar SVD directamente sobre X preserva la precisión original.
10. **Use `np.einsum` para contracciones multidimensionales:** Ofrece la sintaxis más concisa, elimina trasposiciones temporales y minimiza asignaciones de memoria intermedias.

▲ [Regresar al Índice General](#)